

章节 7

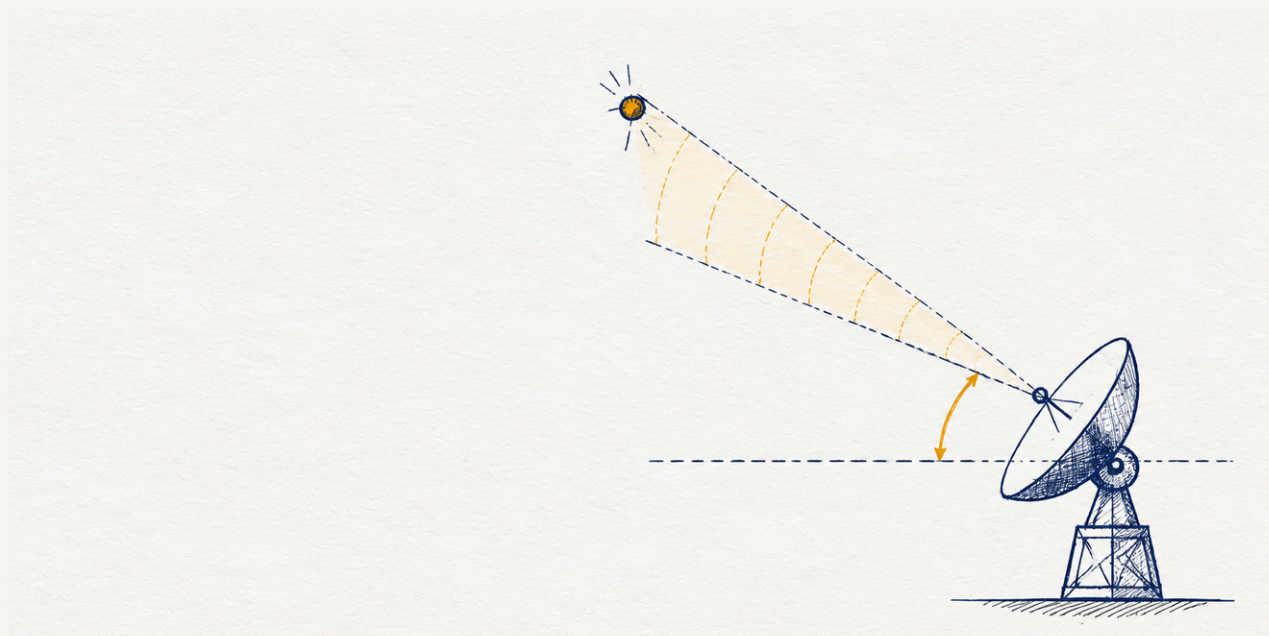
作者：唐承乾

版本：社区版 V2

官方仓库

<https://github.com/apple-art/easy-radar-tutorial>

角度测量



权利声明：本资料为《易懂的雷达信号处理》社区版 V2，仅供个人学习、教学交流与非商业分享使用；作者保留全部著作权，未经授权请勿商用、删改署名或再版传播。

目录

CONTENTS

1. 第 8 章 完整 MATLAB 处理流程	1
1.1. 8.1 仿真条件	1
1.2. 8.2 从回波矩阵到目标表	2
1.3. 8.3 发射信号与匹配滤波器	4
1.3.1. 8.3.1 生成基带 LFM 脉冲	4
1.3.2. 8.3.2 匹配滤波器的作用	5
1.4. 8.4 回波仿真	5
1.4.1. 8.4.1 点目标回波模型	5
1.4.2. 8.4.2 MATLAB 实现	5
1.5. 8.5 距离压缩	6
1.6. 8.6 Doppler 处理	7
1.7. 8.7 检测与目标表生成	8
1.7.1. 8.7.1 阈值检测	8
1.7.2. 8.7.2 从检测点到目标表	8
1.7.3. 8.7.3 目标表的组织	9
1.8. 8.8 参数变化练习	9

前面几章已经把雷达处理链拆开讲过：第 3 章把多发脉冲的 I/Q 样本排成 `rx_matrix`，第 4 章在快时间上做距离处理，第 5 章在慢时间上做 Doppler 处理，第 6 章把距离-速度图上的响应筛成检测结果，第 7 章说明方向信息怎样进入目标参数。

本章把这些步骤放到同一个 MATLAB 程序。程序里每个变量都要回答三个问题：它的维度是什么，下一步沿哪一维处理，处理后对应哪个物理量。

本章采用简化 pulse-Doppler 场景：两个点目标，32 发脉冲，基带 LFM 信号。完整脚本在配套 MATLAB 附件，正文只保留主要片段和检查方法。

1.1. 8.1 仿真条件

本章用两个点目标作为例子。雷达参数延续前面几章的简化设定：使用基带 LFM 脉冲，接收端得到复数 I/Q 样本，多发脉冲按发射顺序组成 `rx_matrix`。

表 1.1 - 雷达参数设置

参数	MATLAB 变量	取值	含义
光速	<code>c</code>	$3 \times 10^8 \text{ m/s}$	距离与时延换算
载频	<code>fc</code>	10 GHz	决定波长
波长	<code>lambda</code>	0.03 m	Doppler 到速度的换算
LFM 带宽	<code>B</code>	10 MHz	影响距离分辨率
脉冲宽度	<code>Tp</code>	$20 \mu\text{s}$	发射脉冲持续时间
采样率	<code>fs</code>	20 MHz	快时间采样间隔
PRF	<code>PRF</code>	5 kHz	慢时间采样率
脉冲数	<code>Npulse</code>	32	决定 CPI 和速度分辨率
快时间采样点数	<code>Nfast</code>	2048	决定接收窗口长度

两个目标的真值设定如下：

表 1.2 - 目标真值设置

目标	距离	径向速度	幅度
A	6 km	$+30 \text{ m/s}$	1.0
B	9 km	-20 m/s	0.75

这里的速度都落在本例的不模糊速度范围内。因为

$$v_{\max} = \frac{\lambda \text{PRF}}{4} = \frac{0.03 \times 5000}{4} = 37.5 \text{ m/s}$$

所以 $+30 \text{ m/s}$ 和 -20 m/s 都能在双边速度轴上显示出来。若把 PRF 降低到 2 kHz , $v_{\max}$ 会变成 15 m/s , 目标 A 就会发生速度模糊。

这里的目标表述有两层。`scene` 里的距离和速度是真值, 用来生成仿真回波; `target_list` 是处理链从回波中估计出来的结果。真实雷达只有后者, 没有前者。

1.2. 8.2 从回波矩阵到目标表

处理链写成一行数据流:

```
scene -> tx -> rx_matrix -> range_data -> rd_map -> det_mask -> target_list
```

各变量的作用和维度如下:

表 1.3 - 处理链各变量的维度与含义

变量	典型维度	轴含义	物理含义
<code>scene</code>	结构体	不适用	目标真值和雷达参数
<code>tx</code>	$1 \times N_t$	快时间	发射基带 LFM 脉冲
<code>rx_matrix</code>	$N_{\text{pulse}} \times N_{\text{fast}}$	行: 脉冲序号; 列: 快时间采样点	原始复数回波矩阵
<code>range_data</code>	$N_{\text{pulse}} \times N_{\text{fast}}$	行: 脉冲序号; 列: 距离相关采样点	距离压缩结果
<code>rd_map</code>	$N_{\text{doppler}} \times N_{\text{range}}$	行: Doppler / 速度 bin; 列: range bin	距离-速度图
<code>det_mask</code>	与 <code>rd_map</code> 相同	行列与 <code>rd_map</code> 对齐	检测掩码
<code>target_list</code>	$N_{\text{target}} \times \text{字段数}$	每行一个目标	距离、速度、响应强度等估计结果

顶层程序组织成下面的形式:

```
scene = ch08_default_scene();
[tx, mf, k] = ch08_make_lfm_pulse(scene.B, scene.Tp, scene.fs);
rx_matrix = ch08_simulate_echo(scene, k);
[range_data, range_axis] = ch08_range_compress(rx_matrix, mf, scene.c, scene.fs);
[rd_map, rd_power, vel_axis] = ch08_doppler_process(range_data, scene);
```

```
[det_mask, row_idx, col_idx] = ch08_detect_targets(rd_power, 0.35);  
target_list = ch08_make_target_list(row_idx, col_idx, rd_power,  
range_axis, vel_axis);
```

这段代码只看函数接口，不看函数内部。第一个输出 `rx_matrix` 还只是接收样本矩阵；最后的 `target_list` 才是可以汇报的目标结果。中间每一步都在改变数据的组织方式。

我们再检查尺寸：

```
size(rx_matrix)  
size(range_data)  
size(rd_map)  
size(target_list)
```

输出应该是：

```
ans =  
    32    2048
```

```
ans =  
    32    2048
```

```
ans =  
    32    2048
```

```
ans =  
     2     3
```

`rx_matrix` 和 `range_data` 的尺寸相同，都是 32×2048 ，因为距离压缩只改变每一行的内容，不改变矩阵尺寸。`rd_map` 的尺寸也是 32×2048 ，因为 Doppler FFT 沿行方向做，输出行数等于 FFT 点数。`target_list` 是 2×3 ，表示检测到 2 个目标，每个目标有 3 个字段（距离、速度、功率）。

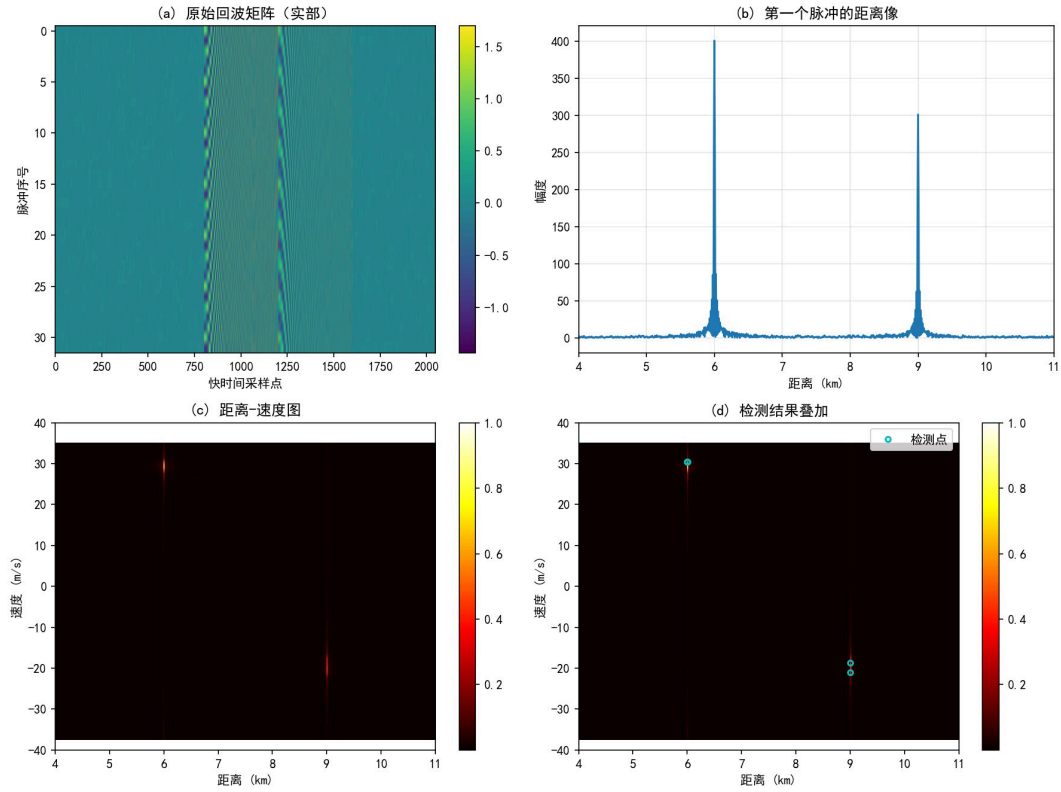


图 8.1 处理链数据流示意图

1.3. 8.3 发射信号与匹配滤波器

1.3.1. 8.3.1 生成基带 LFM 脉冲

第 4 章讲过, LFM 脉冲的瞬时频率随时间线性变化。基带 LFM 信号可以写成:

$$s(t) = \exp(j\pi kt^2), \quad 0 \leq t \leq T_p$$

其中 $k = B/T_p$ 是调频斜率, B 是带宽, T_p 是脉冲宽度。

MATLAB 实现:

```
function [tx, mf, k] = ch08_make_lfm_pulse(B, Tp, fs)
    k = B / Tp;
    t = (0 : 1/fs : Tp - 1/fs);
    tx = exp(1j * pi * k * t.^2);
    mf = conj(fliplr(tx));
end
```

这里 $\mathbf{tx}$ 是发射信号， $\mathbf{mf}$ 是匹配滤波器。匹配滤波器是发射信号的时间反转共轭，用于距离压缩。

1.3.2. 8.3.2 匹配滤波器的作用

匹配滤波器的作用是把接收到的展宽信号压缩成窄脉冲。第 4 章讲过，匹配滤波器的输出峰值位置对应目标的时延，峰值宽度对应距离分辨率。

距离分辨率由带宽决定：

$$\Delta R = \frac{c}{2B}$$

本例中 $B = 10 \text{ MHz}$ ，所以 $\Delta R = 15 \text{ m}$ 。这意味着距离相差 15 m 以上的两个目标可以在距离维上分开。

1.4. 8.4 回波仿真

1.4.1. 8.4.1 点目标回波模型

点目标的回波可以写成：

$$r(t) = A \cdot s(t - \tau) \cdot \exp(j2\pi f_d t)$$

其中 A 是幅度， $\tau = 2R/c$ 是时延， $f_d = 2v/\lambda$ 是 Doppler 频率。

对于多发脉冲，第 m 个脉冲的回波是：

$$r_m(t) = A \cdot s(t - \tau) \cdot \exp(j2\pi f_d (t + mT_{\text{PRI}}))$$

其中 $T_{\text{PRI}} = 1/\text{PRF}$ 是脉冲重复间隔。

1.4.2. 8.4.2 MATLAB 实现

```
function rx_matrix = ch08_simulate_echo(scene, k)
    Npulse = scene.Npulse;
    Nfast = scene.Nfast;
    fs = scene.fs;
    PRF = scene.PRF;
    c = scene.c;
    lambda = scene.lambda;

    rx_matrix = zeros(Npulse, Nfast);
```

```

for i = 1:length(scene.targets)
    tgt = scene.targets(i);
    tau = 2 * tgt.range / c;
    fd = 2 * tgt.velocity / lambda;

    for m = 1:Npulse
        t_pulse = (m-1) / PRF;
        t = (0 : 1/fs : (Nfast-1)/fs);

        phase = 2*pi*fd*(t + t_pulse);
        delayed_signal = tgt.amplitude * exp(1j*pi*k*(t-
tau).^2) .* exp(1j*phase);

        rx_matrix(m, :) = rx_matrix(m, :) + delayed_signal;
    end
end
end

```

这个函数遍历所有目标，对每个目标生成多发脉冲的回波，然后叠加到 `rx_matrix` 中。

1.5. 8.5 距离压缩

距离压缩就是把 `rx_matrix` 的每一行与匹配滤波器做卷积：

```

function [range_data, range_axis] = ch08_range_compress(rx_matrix,
mf, c, fs)
    [Npulse, Nfast] = size(rx_matrix);
    range_data = zeros(Npulse, Nfast);

    for m = 1:Npulse
        range_data(m, :) = conv(rx_matrix(m, :), mf, 'same');
    end

    range_axis = (0 : Nfast-1) * c / (2*fs);
end

```

输出 `range_data` 的每一行是一个距离压缩结果，`range_axis` 是对应的距离坐标轴。

检查距离压缩结果：

```

figure;
plot(range_axis / 1e3, abs(range_data(1, :)));
xlabel('距离 (km)');

```



```
ylabel('幅度');
title('第一个脉冲的距离压缩结果');
```

应该能看到两个峰，分别对应 6 km 和 9 km 的目标。

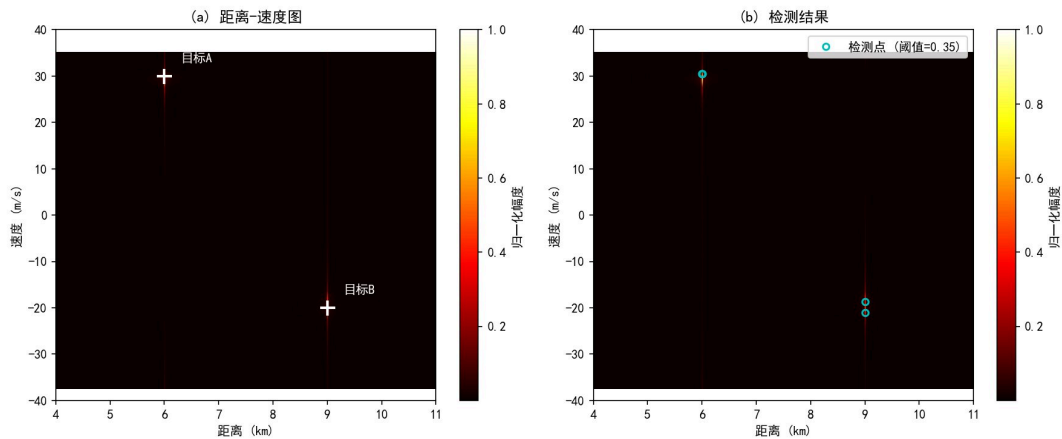


图 8.2 距离压缩结果

1.6. 8.6 Doppler 处理

Doppler 处理是对 `range_data` 的每一列做 FFT:

```
function [rd_map, rd_power, vel_axis] =
ch08_doppler_process(range_data, scene)
    [Npulse, Nfast] = size(range_data);
    rd_map = fftshift(fft(range_data, Npulse, 1), 1);
    rd_power = abs(rd_map).^2;

    PRF = scene.PRF;
    lambda = scene.lambda;
    v_max = lambda * PRF / 4;
    vel_axis = linspace(-v_max, v_max, Npulse);
end
```

输出 `rd_map` 是距离-速度图，`rd_power` 是功率谱，`vel_axis` 是速度坐标轴。

检查距离-速度图:

```
figure;
imagesc(range_axis / 1e3, vel_axis, 10*log10(rd_power));
xlabel('距离 (km)');
ylabel('速度 (m/s)');
title('距离-速度图');
colorbar;
```

应该能看到两个亮点，分别对应 $(6\text{ km}, +30\text{ m/s})$ 和 $(9\text{ km}, -20\text{ m/s})$ 。

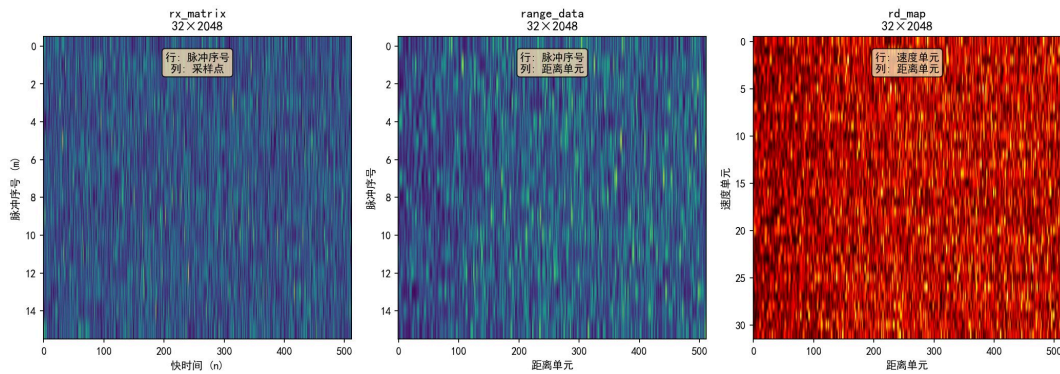


图 8.3 距离-速度图

1.7. 8.7 检测与目标表生成

1.7.1. 8.7.1 阈值检测

检测就是找出 `rd_power` 中超过阈值的点：

```
function [det_mask, row_idx, col_idx] =
ch08_detect_targets(rd_power, threshold)
    max_power = max(rd_power(:));
    det_mask = rd_power > threshold * max_power;
    [row_idx, col_idx] = find(det_mask);
end
```

这里用相对阈值：把最大值乘以一个系数（如 0.35）作为门限。

1.7.2. 8.7.2 从检测点到目标表

检测点的行列索引对应速度和距离：

```
range_est = range_axis(col_idx);
vel_est = vel_axis(row_idx);
```

其中 `range_axis` 对应距离坐标轴，`vel_axis` 对应速度坐标轴。这两个轴在前面的距离压缩和 Doppler 处理中已经生成。

到这一步，前面得到的距离-速度图才真正转化为可以汇报的目标结果。

1.7.3. 8.7.3 目标表的组织

最简单的目标表只保留距离、速度和响应强度：

```
target_list = table(range_est, vel_est,
rd_power(sub2ind(size(rd_power), row_idx, col_idx)), ...
'VariableNames', {'Range', 'Velocity', 'Power'});
```

实际工程中，目标表还会包含检测时刻、信噪比估计、角度信息等字段。第 7 章讲过的角度测量可以在这里并入目标表。

1.8. 8.8 参数变化练习

练习 1: 改变目标距离。把目标 A 的距离从 6 km 改成 7 km ，观察 `range_data` 的距离峰和 `rd_map` 中亮点的横向位置。速度位置不应明显改变。

练习 2: 改变目标速度。把目标 B 的速度从 -20 m/s 改成 $+20\text{ m/s}$ ，观察 `rd_map` 中亮点从负速度一侧移动到正速度一侧。距离位置不应明显改变。

练习 3: 降低脉冲数。把 `Npulse` 从 32 改成 16。速度 bin 变粗，`rd_map` 的速度向分辨能力下降。这个变化对应第 5 章的 CPI 与速度分辨率关系。

练习 4: 降低 PRF。把 PRF 从 5 kHz 改成 2 kHz 。最大不模糊速度变成 15 m/s ，目标 A 的 $+30\text{ m/s}$ 会折叠到 0 m/s 附近，目标 B 的 -20 m/s 会折叠到 $+10\text{ m/s}$ 附近。观察 `vel_axis` 的范围和峰值位置变化。

练习 5: 调整阈值。把 `threshold` 从 0.35 改成 0.2 和 0.6。阈值降低时检测点会增多，阈值升高时弱目标更容易消失。这个练习对应第 6 章的虚警和漏检取舍。

练习 6: 加入一个强零速度杂波。在某个距离附近加入速度为 0 m/s 、幅度较大的回波。距离-速度图上零速度附近会出现强响应，检测模块可能优先报出它。这个练习用来连接第 5 章的零 Doppler 杂波和第 6 章的检测限制。

图 8.4 展示了不同参数设置对处理结果的影响。左列是原始设置，右列分别展示了改变带宽、脉冲数和 PRF 后的效果。

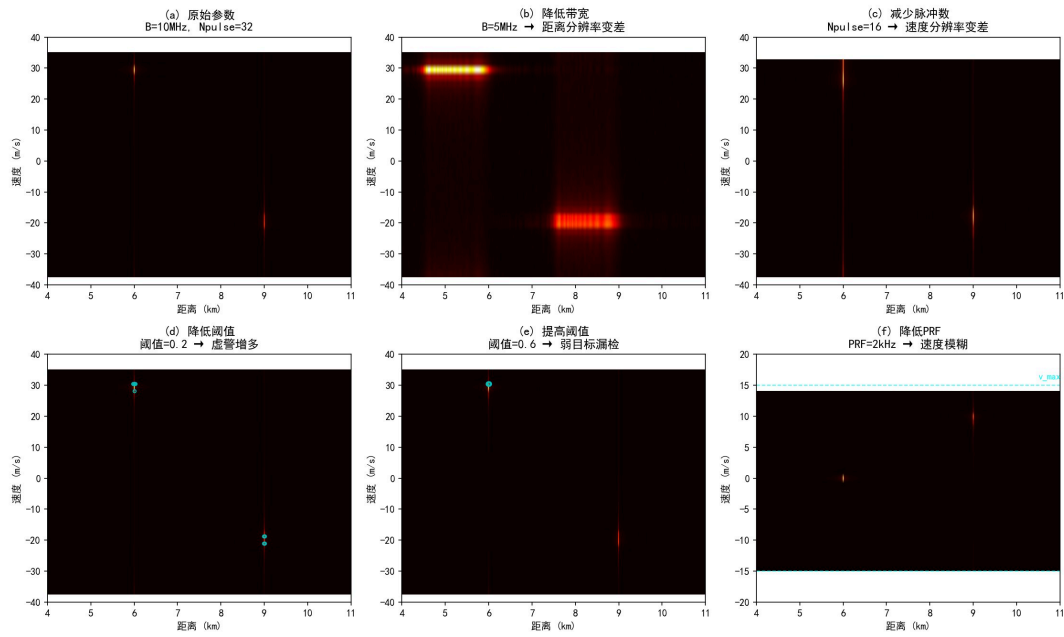


图 8.4 参数变化对处理结果的影响

跑完这些练习后，目标不再只是图上的亮点。它从 `scene` 中的真值出发，经过 `rx_matrix`、`range_data`、`rd_map` 和 `det_mask`，最后变成 `target_list` 中的一行。能解释这条路径，就能把这份 MATLAB 处理流程和前面几章的公式接起来。